Journal of Informatics and Web Engineering
Vol. 3 No. 3 (January 2026) eISSN: 2821-370X

University FAQ Chatbot with Open-source LLMs

HAROLD GOH
[2]Faculty of Computing and Informatics, Multimedia University, Jalan Multimedia, 63100 Cyberjaya, Malaysia
1211108866@student.mmu.edu.my

Abstract – During peak enrollment periods, universities often face challenges in managing high volumes of repetitive admission inquiries. Over the years, universities have adopted traditional information delivery methods including static FAQ websites and manual email responses, which showed an inefficient approach, that leads to student dissatisfaction and increased amound of administrative workload. In this paper, we will discuss the development of an intelligent university FAQ Chatbot that combines Retrieval Augmented Generation (RAG) architecture with open-source Large Language Models (LLMs) to generate accurate and context-aware answers to admission queries.

The proposed system addresses three key limitations of existing solutions: inefficiency in dealing with repetitive questions, less interactivity of information sources, which are static, and lack of contextual accuracy in existing chatbot systems. The implementation of RAG architecture through LangChain framework with Mistral 7B as the generative model makes the system response based on official documents of the university, this significantly reduce hallucinations while maintaining natural language fluency. The chatbot is available in a web-based interface developed using React for the frontend and Flask for the backend.

Through the first phase of requirement analysis using questionnaires and observations, the results shows that 66% of the respondents thinks that getting accurate information with current methods is difficult. The system design demonstrates the overall flow for both users and administrators. The system features include processing queries, maintaining chat history, collecting feedback, and managing the admission's documents. For evaluation, ROUGE-L, BLEU, and METEOR are used to assess models' response accuracy. This research demonstrates the potential of intelligent chatbots that can help to improve efficiency and reduce workload in university admission processes.

Keywords—Chatbot, Retrieval Augmented Generation, Large Language Models, Natural Language Processing, University Admission

Received: 14 June 2025; Accepted: 23 December 2025; Published: 16 January 2026

1. INTRODUCTION

Many universities face challenges ranging from answering replicated inquiries to losing potential students due to slow response. Especially in the digital era, users expect instance, accurate, and quick access to information, yet traditional methods like static FAQs website, manual email admission office, and massive PDF handbook leave many students feeling lost and frustrated. While there is a huge demand for AI chatbots that can automate tasks and

reduce clients' frustrations, the most common problem faced is hallucinations, where models provide out of context responses because they are trained on outdated datasets or lack specialized knowledge. To resolve hallucinations, RAG was introduced to give AI access to a collection of trusted university documents, ensuring the AI extracts most relevant information to answer using the facts provided. LLM is utilized to provide natural language responses that are more interactive and natural. In this project, a University Admission Chatbot that utilizes existing open-source LLM and RAG technology will be developed to assist students in understanding the admission process and to reduce admission staff workloads during peak seasons.

2. LITERATURE REVIEW

2.1 Chatbot in Educational and Service Environment

The classical chatbots from the early phase relied on NLP pipelines, intent classification, and keyword extraction systems. For instance, a research created an admission chatbot to automate responses related to course details that can help Thai university applicants [1]. The study highlight the common issue that chatbots can only respond to questions based on what they have been trained on and tend to miss unstructured or novel queries.

A similar structure characterizes the NLP based chatbot for multiple restaurants, which uses an Artificial Neural Network (ANN) to classify user inputs into predefined intents [2]. The model was built with Dialog Skill which contain the intents, entities, and dialog. Among the three components, intent is classified as the most important as it determines what the user wants and their end goal. On the other hand, stemming, tokenization, and vectorization were used to transform user messages into numeric forms, and ANN was used to predict matching intents. The system may be effective for structured tasks such as restaurant reservations, but it relies on manual creation of training patterns and requires at least five examples for one intent to ensure reliability. This dependency shows the scaling issues in intent based system as domains grow more complex in a classical chatbot design.

The other chatbot introduced multimodality and hybrid AI techniques that increased customer satisfaction in a company's customer service [3]. The study use a combination of NLP and convolutional neural networks (CNNs). The CNNs were used for image classification, while NLP was used to understand and respond to user's queries. The chatbot allows users to upload images or ask questions to get desired information. The system implemented a real-time feedback loops that allow users to flag incorrect answers, these feedbacks were added to the training dataset by human.

To solve the issues in classical intent-based chatbots, RAG was introduced. RAG allows generative AI models to understand context, interpret variety of queries, and produce responses. Intent-based classification, which relies on intent patterns, was replaced in a recent study by with the RAG framework [4]. With RAG approach, it embeds university brochures, websites, and student manuals into vectors and stored into FAISS database. In the retrieval process of RAG, the most relevant textual chunks are selected based on user queries. While the RAG based chatbot shows a huge improvement over earlier intent based systems, but this approach still faces an important challenge of hallucinations.

The problem of hallucinations in AI chatbots is highlighted in another recent study. The study demonstrates that many LLMs continue to generate responses that are contextually or factually incorrect when reading complex documents, despite the fact that RAG improves grounding [5]. In order to solve this, the authors created a local document based chatbot that use optimized open-source LLMs, such as Mistral 7B and LLaMa-2 7B. The system was implemented using LangChain for document processing and handbook content embeddings. For fine-tuning models, a dataset of 228 validated question and answers pairs was used. As a result, the system gives a significant improvement in accuracy compared to non-fine tuned version of the same models.

2.2 Large Language Models (LLMs) overview

The progress of LLMs like GPT-3.5, LLaMa, Mistral, and Phi has provided a massive boost to the overall conversational AI abilities, these models can produce coherent, context-sensitive, natural, and fluent text responses. In education, these capabilities are acted as a cornerstone to assist with tasks such as comprehension, question answering, tutoring, and content retrieval. In a study evaluates the ability of GPT-3.5 Turbo in reading zero-shot prompted Hindi language passages. Besides that, the study highlights the key strength of LLMs which is the ability to generalize different languages even when training data is more toward English [6].

In spite of the advancement of LLMs, there is still exists a common limitation in multilingual and domain specific applications, hallucination, whereby an LLM produces responses that are factually wrong and inconsistent. A work categorized hallucinations into two different categories, intrinsic (hallucinations that occur by manipulating the content of the input) and extrinsic (hallucinations that occur when the output is made and cannot be verified as true or false based on the input) [7].

In another recent study examine how open source LLMs can extract information from unstructured resumes. Although, the study is outside the education domain but its findings are highly relevant as resume data and academic documents, contain different formats and domain specific terminology, which can easily trigger hallucinations. Besides that, the study also compared multiple LLMs and embedding models for structuring education history, skills, and relevant experience. The project demonstrates a critical pattern in which hallucination is often minimized when LLMs are paired with strong embedding models and vector databases [8].

In another work explored the use of LLMs in customer service use case. In this project, it leveraged a hybrid architecture that combines cleaned Twitter customer service dataset with pre-trained LLM embeddings. As a result, the system demonstrated a significant improvement in BLEU score. Moreover, the authors identified that the main problem of hallucination occurs when the model requires additional training data or missing context in the user queries. To address this issue, the proposed study used a combination of prompt engineering, additional training data in the domain, and a vectorized retrieval support. These findings indicate that the LLMs are most effective when grounded with domain data [9].

Finally, another notable approach to resolve hallucinations is multi-stage pipeline that combines knowledge extraction, annotation alignment, vector database storage, and similarity retrieval. In another project, it looks at how LLMs can be practically integrated into educational system. In particular, the project used a multi-state pipeline to transform unstructured educational content into structured knowledge units. As a result, the system significantly improved the precision, recall, F1, and ROUGE-L score compared to a regular LLM with no retrieval support. These results show that the risk of hallucinations can be reduced when data is converted into a structured format [10].

2.3 Retrieval Augmented Generation Architecture

In a recent study, the authors tried to check if fine-tuning of domain specific data based on LLMs could improve the performance of RAG. This study examines the usefulness of fine-tuning in the RAG pipelines, specifically in Biomedical, Legal, and Financial question-answering areas. The study collects datasets from different domains including BioASQ, Australian Open Legal QA, and FinQA, the authors also unified all the data into a structured format that consists of questions, answers, and supporting context. Besides that, the authors evaluated both parameter-efficient fine tuning (LoRA) and full fine tuning on Meta LLaMa 3.1 8B instruct. The result of the evaluation shows that fine-tuning provides an advantage especially in domains that require structured or numerical reasoning. This result suggests that fine-tuning can help RAG to achieve better performance especially when models lack sufficient domain coverage [11].

Beyond that, it is also crucial to dive deep into each RAG components to identify the determinant of RAG effectiveness. The determinant of RAG effectiveness is often identified in the retrieval process. In a traditional RAG systems, embedding similarity between user queries and document chunks can make information weaker, especially when large or heterogeneous collections are involved. In a study, it proposes another retrieval strategy based on question-to-question matching instead of direct query-to-document similarity. The study created a collection of domain specific questions and applied an inverted index matching mechanism. The outcome of this study shows that the retrieval process has an important role in stopping hallucinations and bringing relevance to answers [12].

In educational contexts, RAG systems should balance accuracy, readability, and efficiency. In another recent work attempts to examine how retrieval strategies and prompt engineering can bring impact to RAG performance. The authors further show that structured reasoning prompts such as Chain-of-Thought play a big role in making the model better. Besides that, re-ranking methods were found to be the retrieval enhancement mechanism with the highest effectiveness [13].

A recent research highlights the importance of post generation validation and answer grounding, and it is another safety measure against hallucinations in RAG pipelines. In a research, it presents CA-RAG, which removes the traditional distinction between retrieval and generation. What makes the study different from others is that the authors supply both the query and chunks from the documents to the LLM directly, the system will then use

similarity measures like cosine similarity and dot product to validate the generated responses against the chunks. The study highlights the importance of similarity validation [14].

To solve the privacy issue, another recent work proposed a chatbot that supports college admissions that runs completely on local systems. Instead of using general AI models, the proposed system uses RAG approach that helps to avoid the models from making up answers or hallucinate. The models only answer questions based on official documents that the users provided to the models like handbooks and policies. The system first turns these documents into embeddings using an embedding model, 'nomic-embed-text'. The RAG pipeline starts when a student asks a question, the system uses Gemma 3:4B LLM to generate responses in natural language that human can understand after it finds the most relevant information from the embeddings. The project uses Ollama to run the LLM locally, and Streamlit to design the web interface for users. The study also highlights an improvement technique called context re-ranking, which helps the system choose the most relevant information before generating a response [15].

2.4 Evaluation Metrics and Frameworks

In a study, the authors present an overview of text generation evaluation, focusing on the issues of evaluation for natural language generation tasks such as summarization, dialogue, and text generation. The authors categorize evaluation methods into human centric metrics, untrained automatic metrics like BLEU, ROUGE, and machine learned metrics. In the study, the authors also highlighted the importance of human evaluation though it is expensive and slow. On the other hand, the most used automatic metrics are fast but often fail to reflect meaning, correctness, or factual accuracy [16].

In [17], focus specifically on faithfulness in abstractive summarization. The study defines faithfulness as whether the generated summary can be fully supported by the source document and categorized common factual errors into hallucination. In addition, the authors evaluate several faithfulness metrics like Natural Language Inference (NLI) that check whether summary sentences contained the source, question generation and answering pipelines that verify factuality. The study showed that NLI method correlates better with human judgment than ROUGE, but they are sensitive to model errors and computationally expensive.

As the systems increasingly rely on external knowledge, RAGAS comes in handfuls to extend faithfulness evaluation to RAG systems. The RAGAS framework evaluates RAG systems in these four metrics: context relevance (to evaluate whether retrieved documents are useful), context recall (to evaluate whether information is retrieved), faithfulness (to evaluate whether the answer is grounded) and answer relevance (to evaluate whether the answer addresses the user's question). Instead of using human as a judge, RAGAS relies on large language models prompted as a judge to evaluate each component. This approach gives a fast and scalable evaluation. However, the paper also shows that the results can be vary depending on the prompt wording as well as the model choice, and that RAGAS does not provide a statistical guarantees for its score [18].

To further improve the reliability of RAG evaluation, an automated evaluation framework that is similar to RAGAS was introduced. ARES replaces prompt-only judging with trained LLM based evaluators. Moreover, ARES generates large amounts of synthetic labeled data to train lightweight classifiers for three evaluation tasks, namely, context relevance, answer relevance, and answer faithfulness. This framework then uses a small set of human labeled samples to fine tune the judges and applies Prediction Powered Inference (PPI), this allows the system to report confidence intervals along with the evaluation scores. After multiple experiments with different datasets, the experiments show that ARES achieves higher accuracy with human judgements compared to RAGAS and other metrics [19].

Finally, LLM-as-a-judge method, which generalized the ideas behind systems like RAGAS and ARES. For LLM based evaluation methods, The study proposed three categories including pointwise scoring, pairwise comparison, and ranking based evaluation. These categories analyzed how the models are used across tasks like summarization, code generation, and RAG [20]. In summary, this study highlights the opportunity and threat of LLM based evaluations.

3. METHODOLOGY

The proposed system has a user interface, a backend server, a knowledge base, and a Retrieval Augmented Generation pipeline. The system starts the process by processing the user queries using the retrieval component.

From the university's knowledge base, the retrieval component will try to find for relevant documents based on user's queries, then the retrieved context is passed to the LLMs to generate a response to generate natural languages that human can understand. The overall system architecture is illustrated in Figure 1. The figure shows the complete workflow from submitting student's query through the RAG engine to final response generation, the workflow also include the administrative intervention to maintain the pipeline.

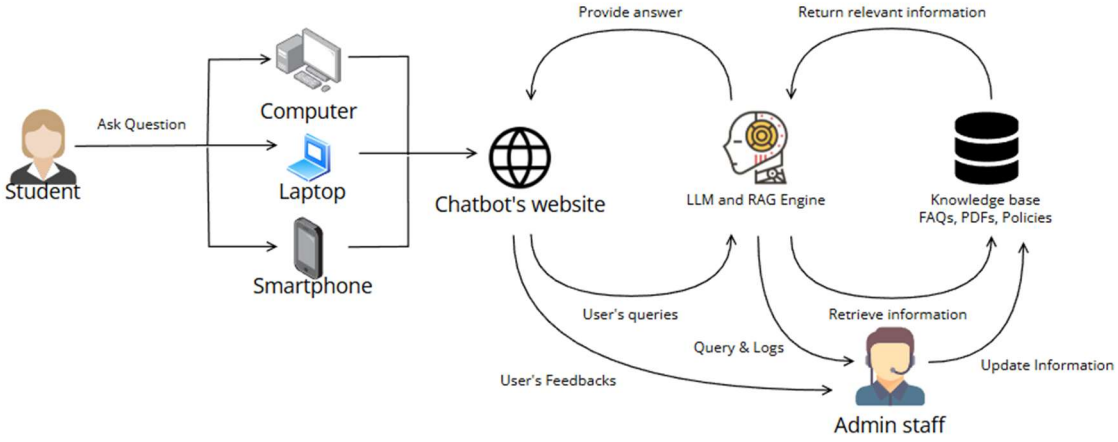

Figure 1. System Architecture

Data preparation is the very first phase of the proposed system. It is through this phase that the system gives reliable results because this phase converts the original documents of a university into vector forms that can be understood by machine learning models. The input data like handbooks, course information, and policy documents are extracted directly from MMU official sites. During the extraction process, information was extracted in its entirety, covering fees and scholarships with numbers of steps in an application, full listed course offerings with criteria, the requirements for admission into different courses, enrollment procedures, facilities on campus, and academic policies and regulations.

After the data extraction, the collected documents are segmented into a smaller chunk. There are two objectives during the chunking process to ensure the optimization of the chunking. First, keeping enough context for the LLM to generate accurate and complete answers, and maintaining retrieval precision by avoiding too broad chunks that affect the relevance. Each processed chunks are then encoded into a vector embedding using the Sentence-BERT model. The 'all-MiniLM-L6-v2' model was selected because of its performance and computational efficiency. The model has showed an advantage in semantic similarity tasks when comparing with traditional word embedding methods like Word2Vec or GloVe. The possible reason behind this performance is because it captures contextual relationships and sentence semantics instead of occurrence patterns. The embeddings are then stored in a vector database, ChromaDB for similarity search. The ChromaDB was chosen because of its performance in high-dimensional nearest neighbor retrieval.

The retrieval and generation phase implements the RAG architecture. The RAG main responsible it to transform the user queries into accurate, contextually grounded responses. To achieve this, the LangChain framework was used to manage the interactions between user input processing, vector database queries, and LLM response generation. This approach provides flexibility for future enhancements or modifying components.

In Figure 2, it illustrates the process when a user submits a query through the web interface.

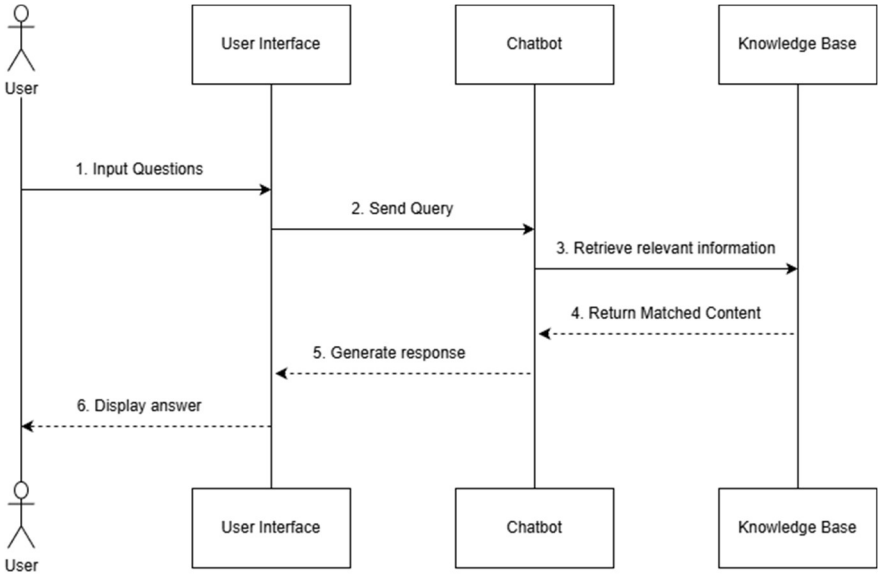

Figure 2. Process of query processing

The vector database performs a k-nearest neighbor search to identify the top-k most similar document chunks. Normally to balance response accuracy with the context window constraints, k=5 is chosen. Similarity score is then calculated using cosine similarity between the query embedding and all the chunk embeddings. The retrieved chunks are then ranked by similarity score, and only the one that exceeds a minimum similarity threshold are retrieved.

The system provides multiple workflows for both the users and administrators. Each designed to address specific needs and responsibilities. In figure 3, it shows the complete user workflow.

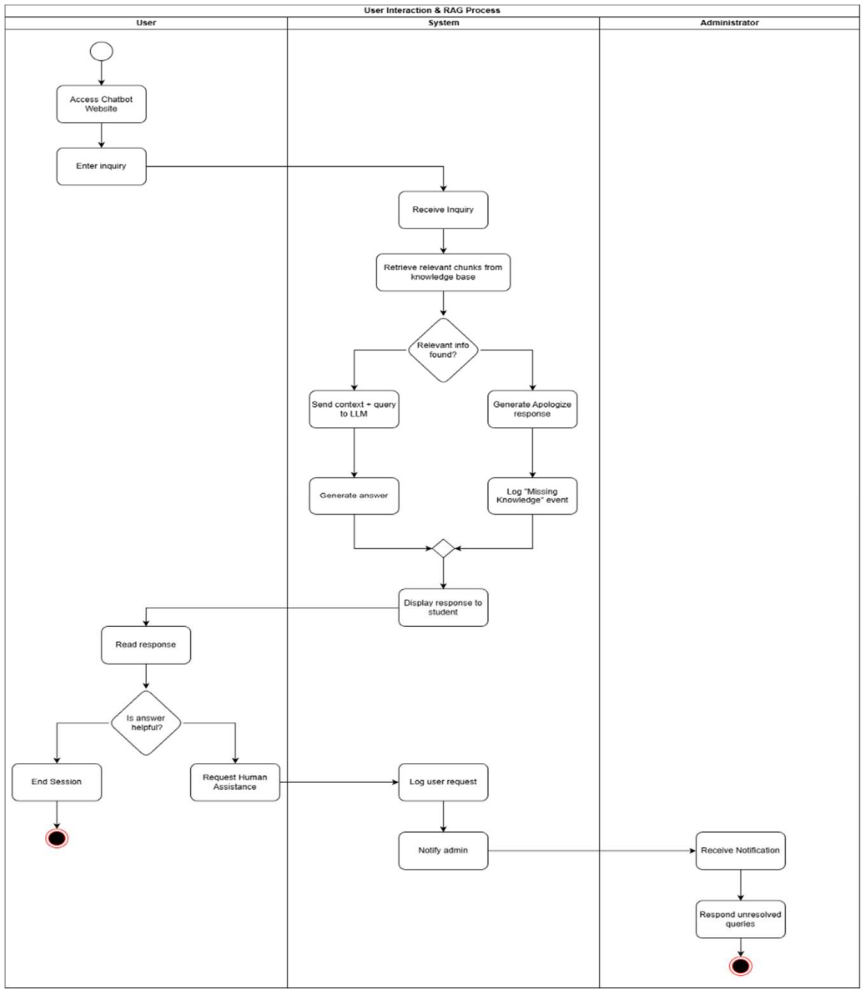

Figure 3. User Interaction Flow

When a student starts an interaction, the system performs authentication on the user first to determines the functionalities that will be available for the user. Users that gain access to personalized features including chat history, save frequently asked questions for quick reference, and request assistance function. On the other hand, users who logged in as a guest user can still interact with the chatbot but with limited features. When submitting a query, the system processes the query and return a response to the user interface.

Users can also provide feedback on response through ratings and comments. The feedback is used to serve as a quality assessment of the system which is used to identify incorrect responses that require administrator attention. Users can request for human assistance, if they face problems when interacting with the chatbot or the chatbot fail to address their needs. This action creates a support ticket for admission staff to review and logs the unsuccessful query for future enhancement.

Figure 4 shows the administrator workflow. It focuses on system maintenance, document management, and monitoring. Administrators have the abilities to upload new documents that are related to the use case to expand the knowledge base, delete irrelevant or outdated information, view query logs that can help identify usage patterns and most asked questions, review user feedback, and check for requests to provide specialized solutions when the chatbot fail to assist specific users needs.

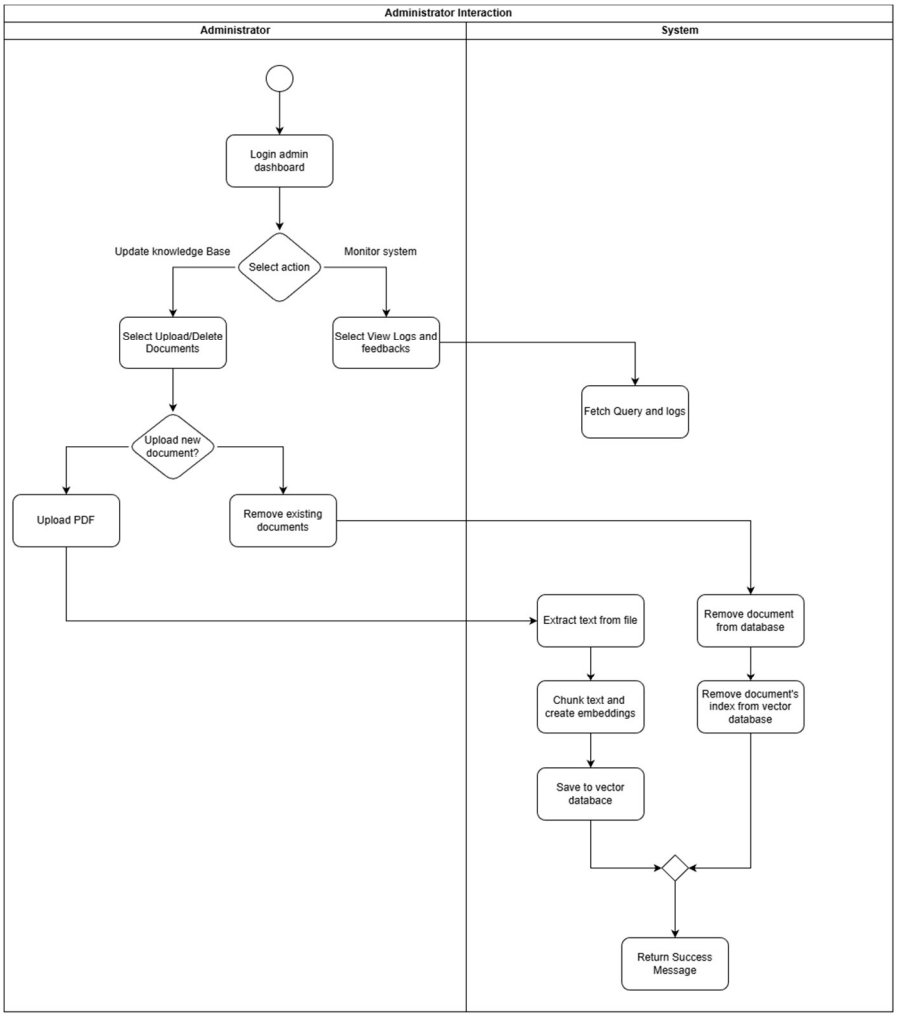

Figure 4. Administrator's Interaction Flow

Administrators are allowed to manage the existing documents or to upload new documents to maintain an up to date knowledge base. When the administrators upload new documents, the system validates file format and size. If the uploaded documents passed the validation, it then uses the initial setup to process the content through the same preprocessing and chunking pipeline. The pipeline generates embeddings for all new chunks and updates the vector database index to include the new information. On the other hand, when the administrators perform deletion, the system will remove both the document and the corresponding embeddings from the database.

The use case diagram in Figure 5 gives an overview of the system features for both primary actors, students and administrators. This diagram identifies all the supported interactions and list out the scope of system functions.

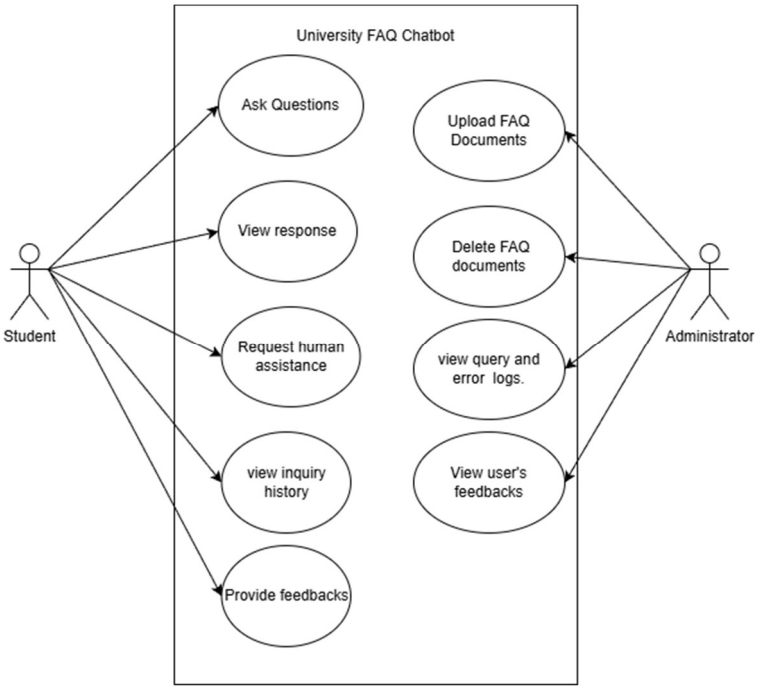

Figure 5. Use Case Diagram

All interactions that facilitate seeking information and providing feedback are included in student use cases. The core functionality of the system, 'Ask Questions' accept natural language queries that is related to MMU admission topic. The 'View Answers' use case format the responses from machine learning model to have appropriate formatting to ensure readability. 'View Chat History' use case allows logged in users to review previous conversations, users can also ask follow up questions or reference to earlier. 'Provide Feedback' use case allows users to rate response based on stars and comments, the users can provide the feedback based on whether the solution provided help address the issues. Finally, 'Request Assistance' use case forward urgent inquiries to admission staff when the provided responses by the chatbot is not helpful.

Next, the administrator use cases focus on system maintenance. 'Upload Documents' use case allow the administrator to extend the knowledge base by adding more sources that are related to the university, this will then trigger processing and reindexing pipeline of the validated documents. 'Delete Documents' use case allows administrators to remove incorrect content. 'View Logs and Query' provides analytics on the system usage like query volume, most common questions, and system's performance. 'View User Feedback' shows user ratings and comments. These administrative use cases allow admin staffs to have more insights of the system and ensure the system remains accurate and responsive to users.

The user interface design focuses on simplicity, accessibility, and responsiveness. The user interface is designed to ensure it provide positive experience to both users and administrator across different devices and technical skills level. To develop a modern interface with smooth interactions, the React framework was utilised. The key components in the interface include the chat window with message history displayed, an input text box with and Enter-to-send functionality, typing indicators, message timestamps, and a feedback button placed with each response to ease the process of providing feedback. Figure 6 shows the interface design for user's main page.

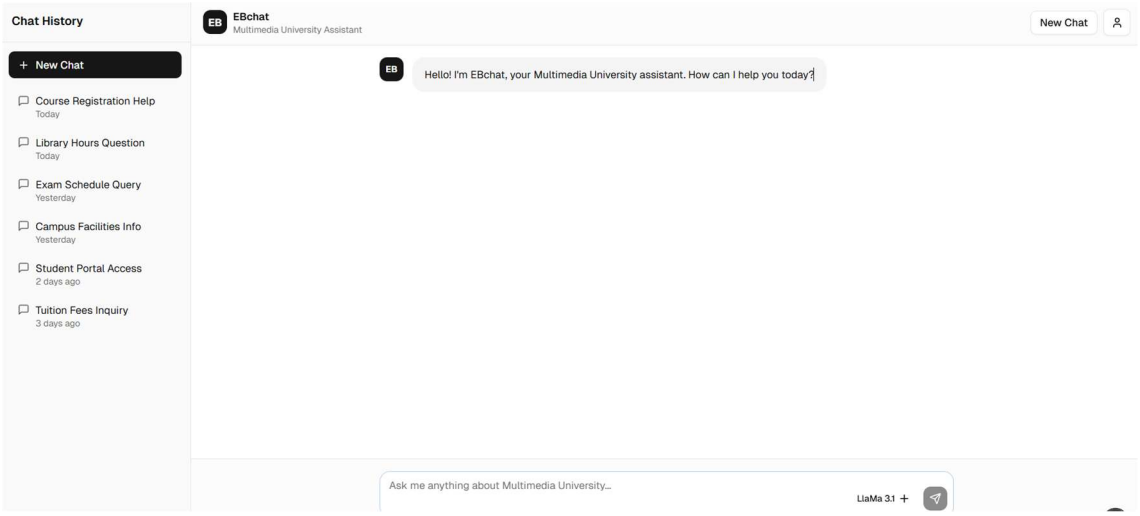

Figure 6. User's Main Page

The system architecture draws a clear separation between the presentation layer (React), logic layer (Flask), and data storage layer (vector database and relational database). This system architecture provides the ability to scale components based on requirements, maintenance can be simplified with isolated changes, and the flexibility to swap implementations.

4. RESULTS AND DISCUSSIONS

Choosing the LLM to utilize in the architecture is an important task, especially in the RAG system design, as the response accuracy and relevance very depend on the capabilities of the chosen LLM. To ensure an informed selection, a comparative assessment of open-source LLMs like Mistral 7B, LLaMa 3.1 8B, and Phi 3.5 was carried out. The three models were selected based on the effectiveness they demonstrated on conversational tasks and a small consumption of computational resources that enable local deployment.

The evaluation approach used the standardized metrics common in LLM assessment. Firstly, ROUGE-L measures the longest subsequence from generated and reference texts. BLEU evaluates n-gram precision and checks how many phrases in generated response are supported by a reference answer. Last, METEOR incorporates stemming, synonyms, and word order, allowing for more comprehensive semantic assessment than BLEU alone.

A test set was curated to include a selection of questions that correspond to various aspects. The 'Reasoning' part tests the ability of the model to perform multi-step logical derivation. Next, the 'Terminology' part tests the capacity of the model to explain the academic terms in plain language. Last, 'Policy' evaluates the understanding of models in the standard university administrative policy. A reference answer was provided for each question to indicate the desired response.

Table 1 and Figure 7 present assessment scores for all three models. Mistral 7B outperformed others on all three metrics, METEOR: 0.3466 shows good performance with both content coverage and semantic accuracy. LLaMa 3.1 8B achieves competitive performance (METEOR 0.3259) as compared to other LLMs, showing strong semantic understanding. Phi 3.5 had lower scores in all metrics, since it is more compact and efficient, this serves as an indication for the trade-off between size of a model and its overall response quality.

Model	ROUGE-L	BLEU	METEOR
Mistral	0.1562	0.0382	0.3466
llama3.1:8b	0.1190	0.0267	0.3259
phi3.5	0.0548	0.0060	0.1699

Table 1: Metric Comparison for Three models

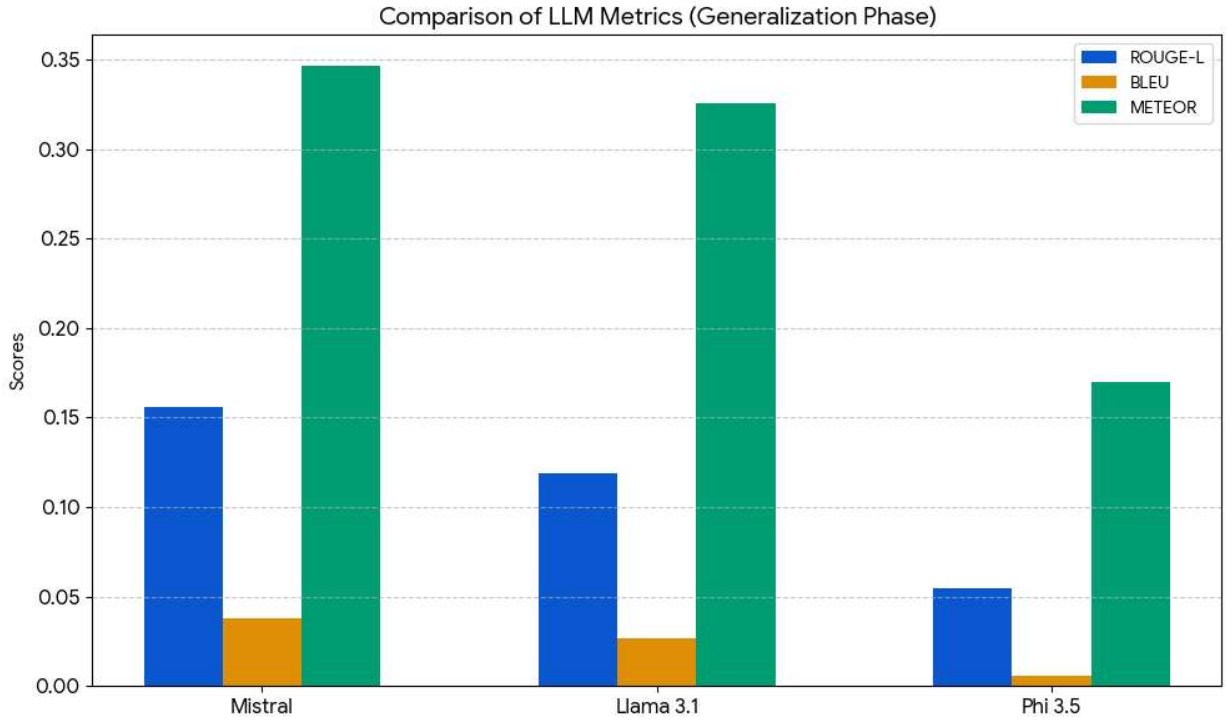

Figure 7. Comparison of LLM Metrics

From the findings above, Mistral 7B was chosen as the main LLM for the University FAQ Chatbot system. The model is well-suited for local deployment in limited resource context. Its high performance across all three metrics evaluation and its 7 billion parameter size, gives a balance between capability and computing requirements. The model's high METEOR score shows a strong semantic understanding of the model, which is essential for producing accurate and relevant answers.

The evaluation code was implemented using Python pandas for results gathering and analysis, the Hugging Face Evaluate module for standardized metric computation, and the Ollama framework for local model inference.

5. CONCLUSION

This study has presented the implementation of a University FAQ Chatbot that uses Retrieval Augmented Generation architecture and open-source Large Language Models. The system addresses common challenges faced in university admission, including how traditional methods handle repetitive inquiries inefficiently, static web pages, and outdated or irrelevant information. Through literature review, the RAG architecture was identified as the optimal approach for grounding LLM responses. While maintaining the conversational fluency, RAG also help to

minimize hallucinations problem. The detailed system design shows the workflows for both students and administrators, flow of processing queries from users, chat history management, feedback collection, and knowledge base maintenance. The implementation using React, Flask, and LangChain frameworks provides a scalable, maintainable foundation for future enhancements. While computational constraints and static knowledge base limitations still has its challenges, the project demonstrates significant potential to enhance information accessibility and operational efficiency in university admission processes.

ACKNOWLEDGEMENT

The authors would like to thank the anonymous reviewers for the suggestions to improve the paper.

FUNDING STATEMENT

The authors received no funding from any party for the research and publication of this article.

AUTHOR CONTRIBUTIONS

CONFLICT OF INTERESTS

No conflict of interests were disclosed.

ETHICS STATEMENTS

This research did not involve animal experiments, or social media data.

DATA AVAILABILITY

The data underlying this study are available in the published article and its online supplementary materials.

REFERENCES

[1] N. Singhuang and P. Natho, "An Enhanced Chatbot for University Admission and Educational Guidance," *IEEE,* 2025.

[2] R. Garg, Riya, S. Thaku, N. Tyagi, N. K. Basha, D. Vij and G. S. Sodhi, "NLP Based Chatbot for Multiple Restaurants," 2021.

[3] R. Vannala, S. B. Swathi and Y. Puranam, "AI CHATBOT FOR ANSWERING FAQ'S," 2022.

[4] D. Joshi, S. Tekade, S. Zanzane, S. Tiwadi, A. Tripathi and D. Sakharwade, "Generative AI-Driven Chatbot for Personalized University Information and Assistance," 2024.

[5] D. I. Rachman, A. S. Akbar and A. D. Sabilla, "Academic Guidebook Chatbot: Performance Comparison of Fine-Tuned Mistral 7B and LlaMa-2 7B," 2025.

[6] A. Jha, P. Mann, A. Tiwari, J. Singh, A. Sharma and K. Kadian, "Evolving Education in India: Multilingual LLMs for," 2024.

[7] G. P. Reddy, Y. V. P. Kumar and K. P. Prakash, "Hallucinations in Large Language Models," 2024.

[8] A. Atiq, G. Ubakanma and M. Iqbal, "Evaluating efficacy of LLMs and Embedding Models in Resume Parsing: A Comparative Analysis Using Open Source LLMs," 2025.

[9] F. Shareef, "Enhancing Conversational AI with LLMs for Customer Support Automation," 2024.

[10] Y. Gong, H. Liu, X. Lian and G. Ma, "Research on Educational Knowledge Base Question Answering System Based on Large Language Model," 2024.

[11] A. Raj, "Fine-tuning Large Language Models with Domain-Specific Data for Retrieval-Augmented Question-Answering Systems," 2025.

[12] B. Saha, U. Saha and M. Z. Malik, "QuIM-RAG: Advancing Retrieval-Augmented Generation With Inverted Question Matching for Enhanced QA Performance," *IEEE,* 2024.

[13] C. C. Y. Yang, G.-J. Liu and C. J. Yu, "Optimizing RAG in Programming Education: A Comparative Study of Models and Strategies," *IEEE,* 2025.

[14] E. Collini, F. I. Kurniadi, P. Nesi and G. Pantaleo, "Context-Aware Retrieval Augmented Generation Using Similarity Validation to Handle Context Inconsistencies in Large Language Models," *IEEE,* 2025.

[15] V. Yadav, A. Birje and G. Megha, "Rag Based Personal AI Chatbot for College," 2025.

[16] A. Celikyilmaz, E. Clark and J. Gao, "Evaluation of Text Generation: A Survey," 2021.

[17] T. Fischer, S. Remus and C. Biemann, "Measuring Faithfulness of Abstractive Summaries," in *KONVENS*, 2022.

[18] S. Es, J. James, L. Espinosa-Anke and S. Schockaert, "RAGAS: Automated Evaluation of Retrieval Augmented Generation," 2024.

[19] J. S. Falcon, O. Khattab, C. Potts and M. Zaharia, "ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems," 2024.

[20] J. Gu, X. Jiang, Z. Shi, H. Tan, X. Zhai and C. Xu, "A Survey on LLM-as-a-Judge," 2025.

BIOGRAPHIES OF AUTHORS
